

KLv7m7q7 → classroom code
anusha@ubut.cse.edu

Python → Interprets

Compiler translates the entire code at once.

Interpreter translates the code line by line.

Language Processing
System (LPS)

HLL - High Level Language

HLL/Source Code

Pre processor

Pure HLL / Preprocessed code/
Modified source code

Compiler

Target Assembly Code

Assembler

Relocatable Macro Code/
Machine Code

Linker

Loader

Executable code

Memory

Example Code

Shayan

```
#include <stdio.h>
#define x 10
int main()
{
    int y = 20, z;
    z = x + y;
    return 0;
}
```

① This goes to the preprocessing system

↳ File inclusion: removing #include <...>

↳ Macro processing: removing #define var

output: Modified source code

② This goes to the compiler that compiles it to the target assembly code



③ This goes to the Assembler

④ Linker

Produce 'obj' file 'Machine Code'
Object file

↳ Links

includes & macros

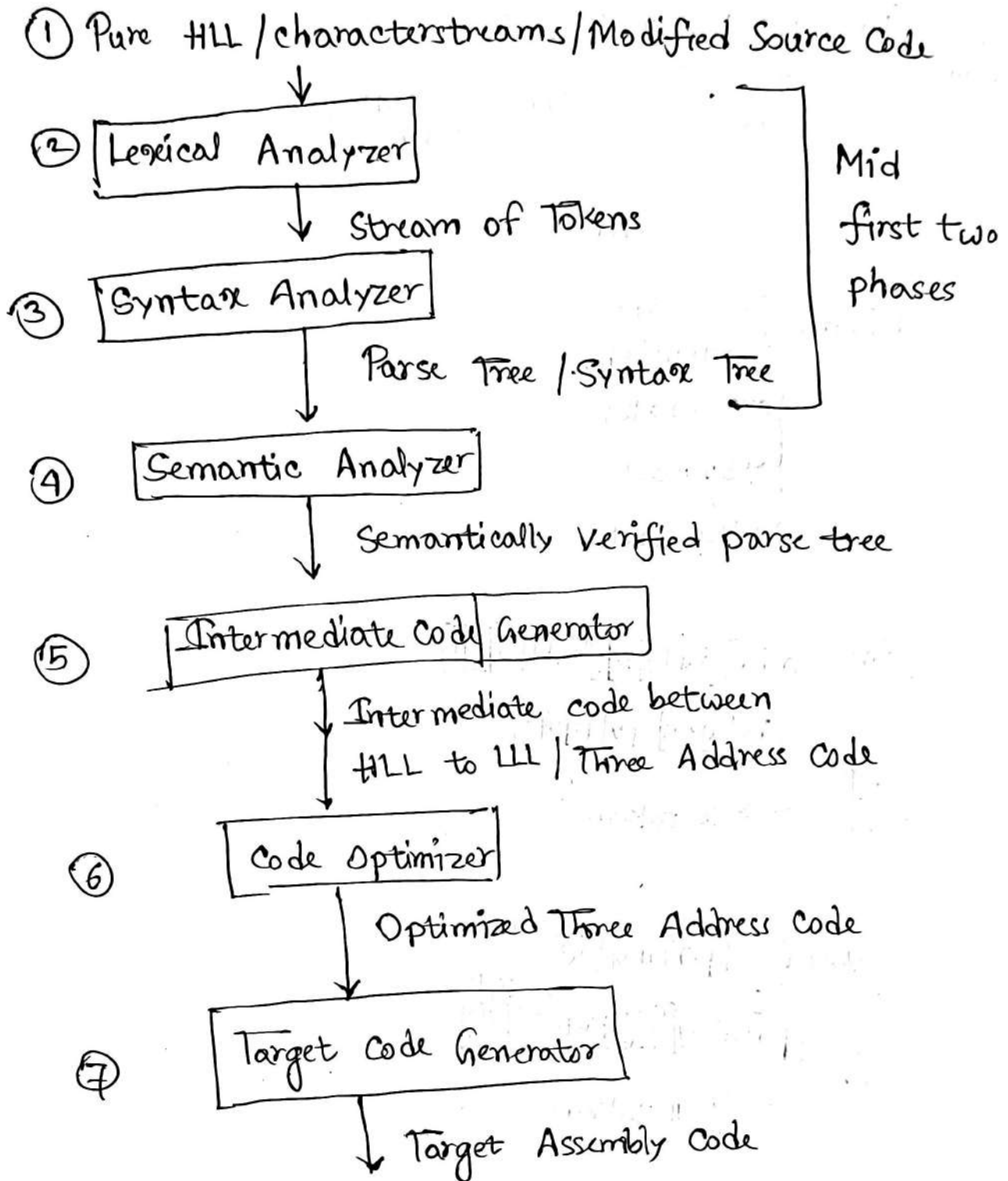
→ ⑤ Goes to the loader that produces exe

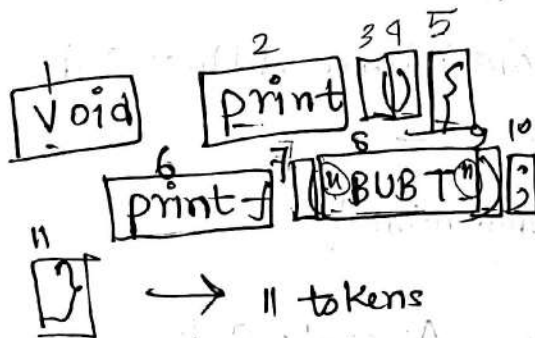
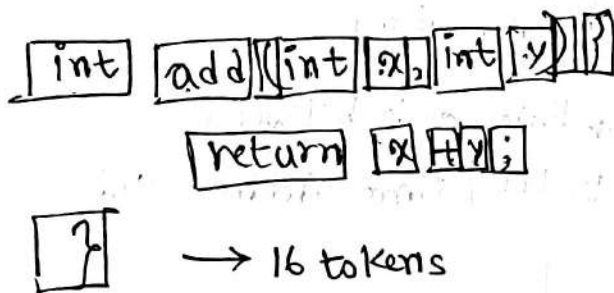
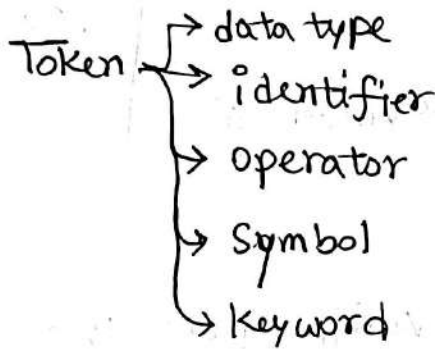
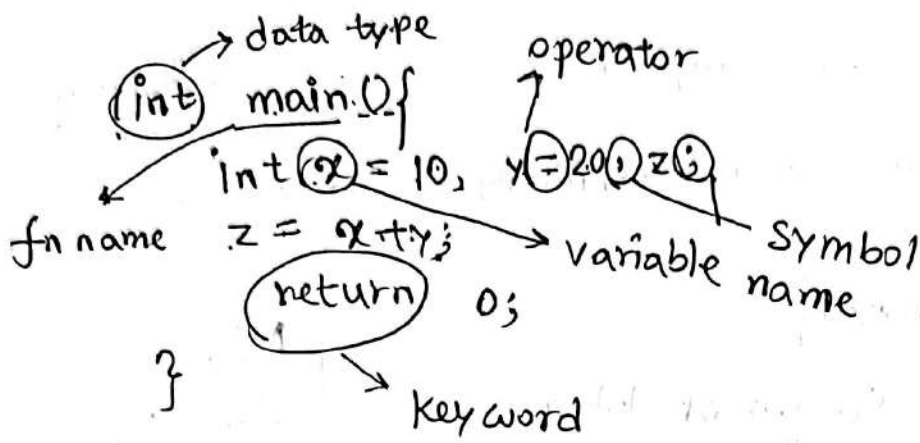
Executes
in the memory

②

Phases of Compiler

Shayan





Types of Error

Shayan

- ① Lexical ~~Analyzer~~ Error : वाक्य शुद्ध अक्षरों में errors
- ② Syntax ~~Analyzer~~ Error : Symbol/operator " "
- ③ Symantic ~~Analyzer~~ Error : Type casting Error/
Logical Error/
Declaration Error

① Lexical Error

inf x = 10;
Lexical error

int 1x = 10;
Lexical error

int @x = 10;

② Syntax Error

int a, b, c ;
c = a [] b []
 ↑ missing operator
 missing symbol

③ Symantic Error

if (a = 10)
int add (int x, int y) {
 float c = x + y;
 return 10.0;
}

Symantic error

int main() {
 hello();
}

error: Declaration error
Symantic Error

⑤

Lexical Lexical

```

int @ add(int 1a, int b)
{
    semantic
    float c = 10;
    semantic ← d = 1a + b; → lexical
    float z;
    semantic print("%d", d);
    semantic/lexical ← return z;
}
    semantic

```

Next Class:

Compiler Phases

Examples

return

CE 323

Class-02

11-May-2026

Shayan

Previous: Lexical Error
Semantic Error
Syntax Error } Types of Error

Phases of Compiler

Today: Example of phases of Compiler

Translation of a Statement using
Compilation Phases

float result, a, b, c;

result = (a/10) * (b+c);

STATEMENT

Lexical Analyzer:

'id1' '=' '(' 'id2' '/' '10' ')' '*' '(' 'id3' '+' 'id4' ')'

stream of tokens

Syntax Analyzer:

— Binary Tree should be used

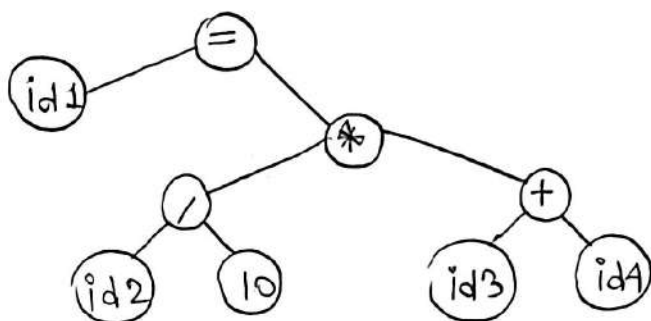


Figure: Syntax Tree or
Parse Tree

Semantic Analyzer:

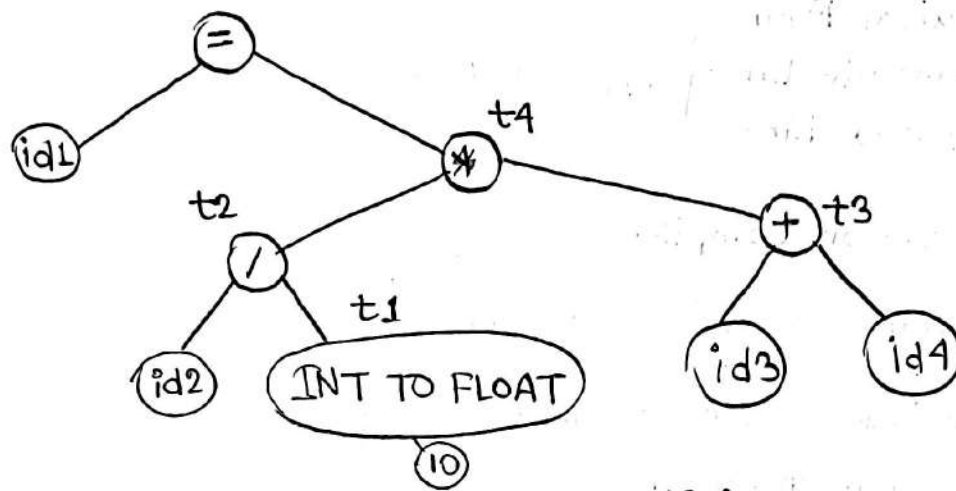


figure: Semantically verified
Parse Tree

Intermediate Code Generator:

$t1 = \text{INT TO FLOAT}$ Max operands = 3
 $t2 = \text{id2} / t1$
 $t3 = \text{id3} + \text{id4}$
 $t4 = t2 * t3$
 $\text{id1} = t4$

Code Optimizer:

$t2 = \text{id2} / 10.0$
 $t3 = \text{id3} + \text{id4}$
 $\text{id1} = t2 * t3$

Shayan

Target Code Generator:

সব identifier কে REGISTER-এ নিতে হবে

MOV R1, id2 # R1 = id2
dst source

DIV R1, #10.0 ; R1 = R1/10.0

MOV R2, id3 ; R2 = id3

MOV R3, id4 ; R3 = id4

ADD R2, R3 ; R2 = R2 + R3

MUL R1, R2 ; R1 = R1 * R2

MOV id1, R1 ; id1 = R1

Priority

ভাগ

গুন

যোগ

বিয়োগ

Shayan

float result, a, b, c, d, e, f, i, g;

result = (a+b*20) + (c/d) * (e-f) / (i+g)

STATEMENT

Lexical Analyzer:

'id1' '=' '(' 'id2' '+' 'id3' '*' '20' ')' '+' '(' 'id4' '/' 'id5' ')' '*' '(' 'id6' '-' 'id7' ')' '/' '(' 'id8' '+' 'id9' ')'

Stream of tokens

Syntax Analyzer:

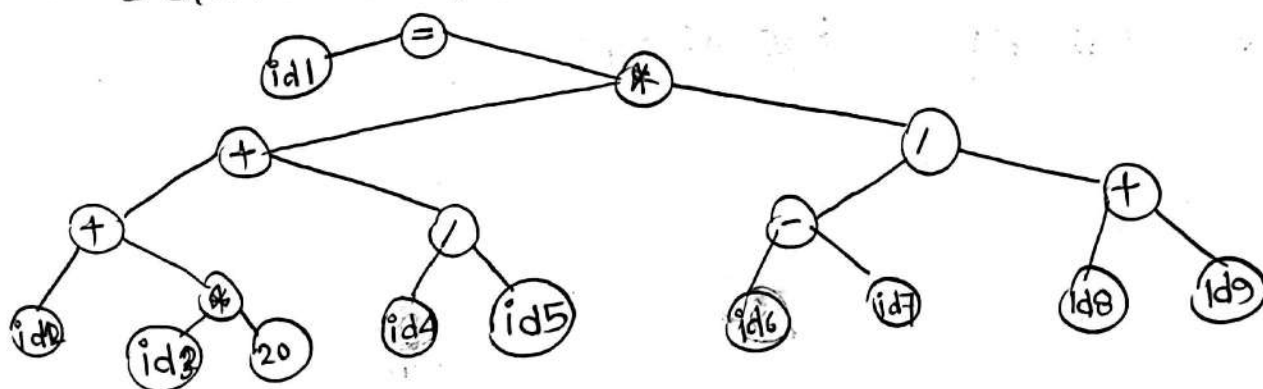


figure: Syntax Tree

Semantic Analyzer:

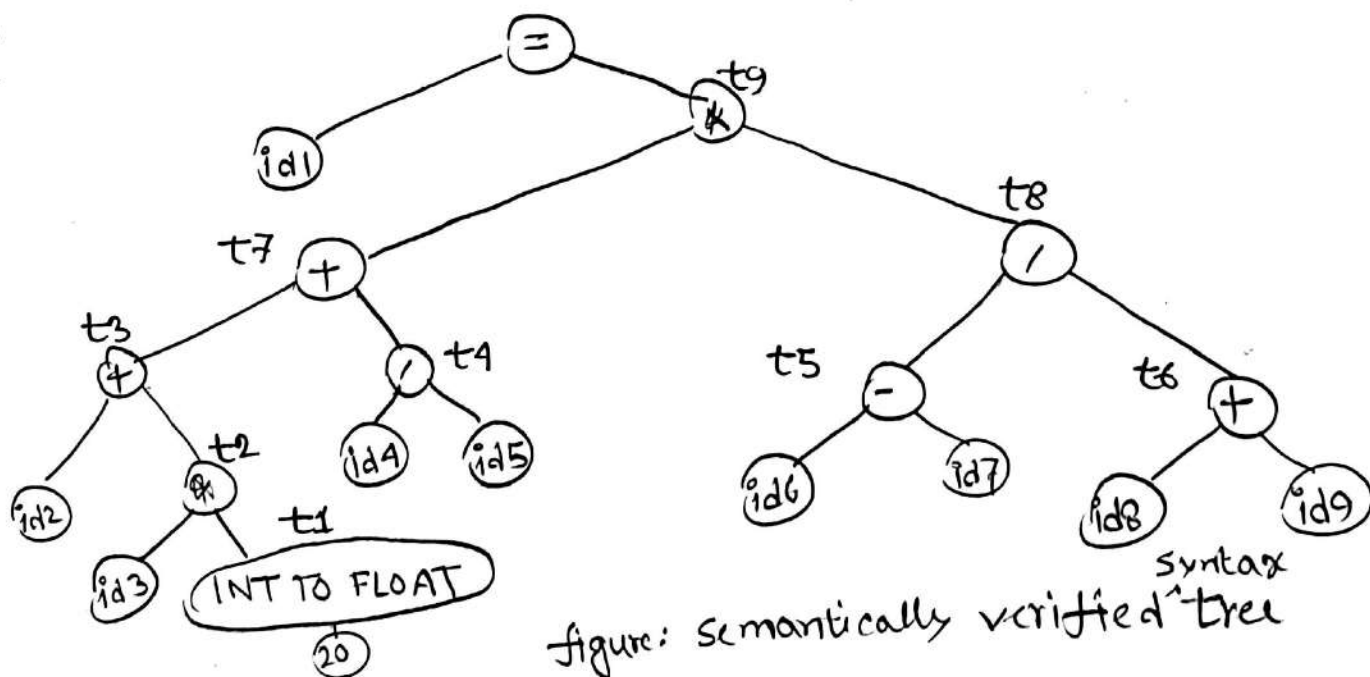


figure: semantically verified tree

may an

Intermediate Code Generator:

$t1 = \text{INT TO FLOAT}$

$t2 = id3 * t1$

$t3 = id2 + t2$

$t4 = id4 / id5$

$t7 = t3 + t4$

$t5 = id6 - id7$

$t6 = id8 + id9$

$t8 = t5 + t6$

$t9 = t7 * t8$

$id1 = t9$

Optimized Code should
Contain 3 register
Max will be allowed
up to 5

Code Optimizer:

$t2 = id3 * 20.0$

$t3 = id2 + t2$

$t4 = id4 / id5$

$t7 = t3 + t4$

$t5 = id6 - id7$

$t6 = id8 + id9$

$t8 = t5 + t6$

$id1 = t7 * t8$

Target Code Generator:

MOV R1, id3 ; R1 = id3

MUL R1, #20.0 ; R1 = 20.0

MOV R2, id2 ; R2 = id2

ADD R2, R1 ; R2 += R1

MOV R3, id4

MOV R4, id5

~~DIV~~ ~~ADD~~ R3, R4 ; ~~R3 /= R4~~ ^{R3 /= R4}

ADD R3, R4 ; R3 += R4

MOV R5, id6

MOV R6, id7

~~SUB~~ ^{R5} R5, R6 ; R5 -= R6

MOV R7, id8

MOV R8, id9

ADD R7, R8

ADD R5, R7 | MOV id1, R5

(5)